Visual-RAG Project Report

Interactive retrieval-augmented generation dashboard for exploring semantic search over NASA Solar System documents.

Git repository: https://github.com/VictorRadelytskyi/Visual-RAG.git

100	58,820	187	6
source documents	raw corpus words	semantic chunks	content categories

Executive Summary

Visual-RAG is a compact end-to-end AI engineering project that makes retrieval behavior visible. It starts with NASA Solar System articles, cleans and chunks the text, embeds each chunk with a sentence-transformer model, stores vectors in a FAISS index, and exposes the retrieval results through an interactive Streamlit interface. The application can run as a retrieval-only explorer or, when Azure OpenAI credentials are configured, generate grounded answers from the retrieved context.

Project Objectives

- Demonstrate a complete document-oriented RAG workflow from raw source collection to query-time retrieval.
- Show how embedding-space projections can help users inspect clusters, nearest neighbors, and weak retrieval cases.
- Provide a usable Streamlit dashboard rather than only a command-line or notebook prototype.
- Keep answer generation optional so the core retrieval system remains testable without cloud credentials.

System Architecture

Layer	Implementation	Role
Corpus ingestion	src/download_corpus.py	Fetches NASA pages, extracts article text, cleans whitespace, and stores raw documents plus metadata.
Chunking	src/chunking.py	Splits each document into overlapping word windows using 220-word chunks with 40-word overlap.
Embedding and index	src/indexing.py	Encodes chunk text with sentence-transformers/all-MiniLM-L6-v2 and builds a FAISS inner-product index.
Projection	src/projection.py	Creates PCA, Truncated SVD, t-SNE, and UMAP two-dimensional views for visual analysis.

Layer	Implementation	Role
Retrieval	src/retrieval.py	Embeds the query, searches FAISS, and returns ranked chunks with metadata and similarity scores.
Interface	app.py and pages/1_Chunk_Detail.py	Provides query controls, Plotly maps, retrieved chunk tables, text inspection, and optional answer generation.

Data and Index Profile

The current processed build contains 187 chunks from 100 NASA source documents. The embedding model recorded in the processed artifacts is **sentence-transformers/all-MiniLM-L6-v2**. Available projection methods are UMAP.

Category	Documents	Chunks
dwarf_planet	2	14
moon	75	56
overview	3	16
planet	16	69
small_body	2	18
star	2	14

User Workflow

- Run python -m scripts.build_all to refresh downloaded documents, chunks, embeddings, FAISS index, and projections.
- Start the interface with streamlit run app.py.
- Enter a natural-language Solar System question and select the number of nearest chunks.
- Inspect the full embedding map, the retrieved-only map, ranked chunk table, and expanded source text.
- Optionally configure Azure OpenAI environment variables to generate a concise answer grounded in the retrieved chunks.

Strengths

- Clear separation between ingestion, chunking, indexing, projection, retrieval, generation, and interface code.
- Retrieval stays explainable because users can inspect both the ranked chunks and their placement in embedding space.
- The app degrades gracefully into retrieval-only mode when Azure OpenAI credentials are absent.
- The project uses practical, recognizable tools: Streamlit, FAISS, sentence-transformers, Plotly, and Azure OpenAI.

Limitations and Next Steps

- Add automated retrieval evaluation queries with expected source documents or answer-bearing chunks.
- Persist richer provenance in the UI, including direct NASA source links for each displayed chunk.
- Consider hybrid retrieval or reranking for vague queries where semantically similar objects compete.
- Add deployment documentation and lightweight screenshots for portfolio or classroom review.

Repository

The complete project source is available at https://github.com/VictorRadelytskyi/Visual-RAG.git.